

Actividad guiada — Ionic + Angular Standalone: consumo de API REST

DAM2 – Desarrollo de Interfaces

Objetivo: crear una aplicación Ionic con Angular Standalone que consuma <https://dummyjson.com/products>, renderice los productos en una tabla, gestione la navegación mediante rutas, publique el proyecto en GitHub usando la rama [desarrollo](#) y lo despliegue en Vercel.

Resultado final

La aplicación tendrá dos vistas:

/inicio

/productos

Con esta estructura:

```
src/
├── app/
│   ├── models/
│   │   └── product.model.ts
│   ├── services/
│   │   └── product.service.ts
│   ├── pages/
│   │   ├── inicio/
│   │   │   └── inicio.page.ts
│   │   └── productos/
│   │       └── productos.page.ts
│   ├── app.component.ts
│   ├── app.routes.ts
│   └── app.config.ts
```

Crear el proyecto

Partimos de una instalación actual de Ionic CLI:

```
npm install -g @ionic/cli
```

Creamos el proyecto:

```
ionic start dam2-productos blank --type=angular
```

Entramos:

```
cd dam2-productos
```

Y arrancamos:

```
ionic serve
```

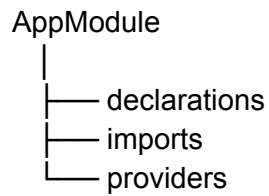
Comprobamos que funciona:

```
http://localhost:8100
```

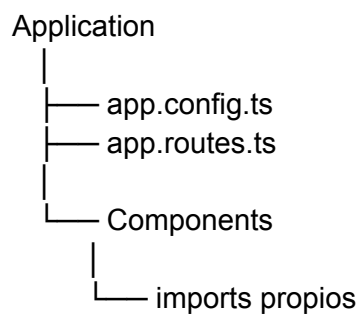
¿Qué significa Standalone?

Antes de empezar a programar, introducimos el concepto.

En Angular tradicional teníamos:



En Angular Standalone:



Por ejemplo, una página puede indicar directamente:

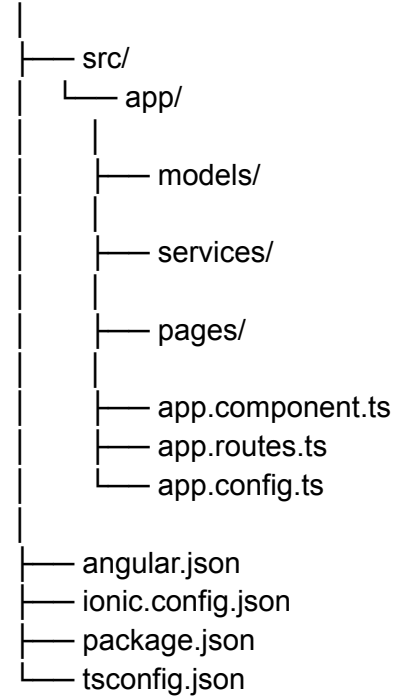
```
@Component({
  standalone: true,
  imports: [
    CommonModule,
    IonicModule
  ]
})
```

No necesita estar declarada en ningún **NgModule**.

Estructura inicial

El proyecto debería evolucionar hacia:

dam2-productos/



Configurar HttpClient en Standalone

Este punto cambia respecto a Angular basado en módulos.

Abrimos:

src/app/app.config.ts

Importamos:

```
import { ApplicationConfig } from '@angular/core';  
import { provideHttpClient } from '@angular/common/http';
```

Y configuramos:

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideHttpClient()  
  ]  
};
```

La idea importante para el alumnado es:

En Standalone no añadimos `HttpClientModule` a un `imports` de `AppModule`.

Utilizamos:

```
provideHttpClient()
```

Angular documenta `provideHttpClient()` como la forma de proporcionar `HttpClient` en la configuración de la aplicación.

Crear el modelo

Creamos:

```
src/app/models/product.model.ts
```

```
export interface Product {
```

```
  id: number;
```

```
  title: string;
```

```
  description: string;
```

```
  category: string;
```

```
  price: number;
```

```
  discountPercentage: number;
```

```
  rating: number;
```

```
  stock: number;
```

```
  brand: string;
```

```
  thumbnail: string;
```

```
}
```

```
export interface ProductsResponse {
```

```
  products: Product[];
```

```
  total: number;
```

```
  skip: number;
```

```
  limit: number;
```

```
}
```

Aquí estamos describiendo la respuesta de:

<https://dummyjson.com/products>

La API devuelve los productos dentro de:

`response.products`

Crear el servicio

Ejecutamos:

```
ionic generate service services/product
```

Tendremos:

```
src/app/services/product.service.ts
```

El servicio será:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

import { ProductsResponse } from '../models/product.model';

@Injectable({
  providedIn: 'root'
})
export class ProductService {

  private http = inject(HttpClient);

  private apiUrl = 'https://dummyjson.com/products';

  getProducts(): Observable<ProductsResponse> {
    return this.http.get<ProductsResponse>(this.apiUrl);
  }
}
```

Importante

Estamos usando:

```
private http = inject(HttpClient);
```

en lugar de:

```
constructor(private http: HttpClient) {}
```

Ambas formas son válidas, pero podemos aprovechar la práctica para introducir la API `inject()` de Angular.

Crear las páginas como Standalone

Aquí también debemos ser cuidadosos.

Podemos generar:

```
ionic generate page pages/inicio
```

y:

```
ionic generate page pages/productos
```

Pero el alumno debe comprobar que las páginas son **Standalone**.

Una página debe tener una estructura similar a:

```
@Component({  
  selector: 'app-productos',  
  templateUrl: './productos.page.html',  
  styleUrls: ['./productos.page.scss'],  
  standalone: true,  
  imports: [  
    CommonModule,  
    IonicModule  
  ]  
})
```

No debe existir:

```
@NgModule(...)
```

Routing Standalone

Abrimos:

src/app/app.routes.ts

Configuramos:

```
import { Routes } from '@angular/router';

export const routes: Routes = [

  {
    path: "",
    redirectTo: 'inicio',
    pathMatch: 'full'
  },

  {
    path: 'inicio',
    loadChildren: () =>
      import('./pages/inicio/inicio.page')
        .then(m => m.InicioPage)
  },

  {
    path: 'productos',
    loadChildren: () =>
      import('./pages/productos/productos.page')
        .then(m => m.ProductosPage)
  },

  {
    path: '**',
    redirectTo: 'inicio'
  }

];
```

Configurar **app.component.ts**

En una aplicación Standalone, el componente raíz también es Standalone.

La estructura debe ser similar a:

```
import { Component } from '@angular/core';
import { IonApp, IonRouterOutlet } from '@ionic/angular/standalone';

@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  imports: [
    IonApp,
    IonRouterOutlet
  ]
})
export class AppComponent {

}
```

app.component.html

Utilizaremos:

```
<ion-app>

  <ion-router-outlet></ion-router-outlet>

</ion-app>
```

El `ion-router-outlet` será el lugar donde Ionic renderizará las páginas correspondientes a la ruta actual.

Página Inicio Standalone

src/app/pages/inicio/inicio.page.ts

Podemos utilizar:

```
import { Component } from '@angular/core';
import {
  IonHeader,
  IonToolbar,
  IonTitle,
  IonContent,
  IonButton
} from '@ionic/angular/standalone';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-inicio',
  templateUrl: './inicio.page.html',
  styleUrls: ['./inicio.page.scss'],
  standalone: true,
  imports: [
    IonHeader,
    IonToolbar,
    IonTitle,
    IonContent,
    IonButton,
    RouterLink
  ]
})
export class InicioPage {

}
```

Aquí aparece una de las características fundamentales de Standalone:

```
imports: [  
  IonHeader,  
  IonToolbar,  
  IonTitle,  
  IonContent,  
  IonButton,  
  RouterLink  
]
```

El componente declara exactamente lo que utiliza.

HTML de Inicio

`inicio.page.html`:

```
<ion-header>  
  <ion-toolbar>  
    <ion-title>  
      Inicio  
    </ion-title>  
  </ion-toolbar>  
</ion-header>  
  
<ion-content class="ion-padding">  
  
  <h1>DAM2 - API REST</h1>  
  
  <p>  
    Aplicación Ionic + Angular Standalone  
    para consumir una API REST.  
  </p>  
  
  <ion-button routerLink="/productos">  
    Ver productos  
  </ion-button>  
  
</ion-content>
```

Página Productos Standalone

productos.page.ts:

```
import { Component, OnInit, inject } from '@angular/core';
```

```
import {  
  IonHeader,  
  IonToolbar,  
  IonTitle,  
  IonContent,  
  IonButtons,  
  IonBackButton,  
  IonSpinner,  
  IonCard,  
  IonCardHeader,  
  IonCardTitle,  
  IonCardContent,  
  IonButton  
} from '@ionic/angular/standalone';
```

```
import { CommonModule } from '@angular/common';
```

```
import { Product, ProductsResponse }  
  from '../models/product.model';
```

```
import { ProductService }  
  from '../services/product.service';
```

```
@Component({  
  selector: 'app-productos',  
  templateUrl: './productos.page.html',  
  styleUrls: ['./productos.page.scss'],  
  standalone: true,  
  imports: [  
    CommonModule,  
  
    IonHeader,  
    IonToolbar,  
    IonTitle,  
    IonContent,  
    IonButtons,  
    IonBackButton,  
    IonSpinner,  
    IonCard,  
    IonCardHeader,  
    IonCardTitle,  

```

```

        IonCardContent,
        IonButton
    ]
})
export class ProductosPage implements OnInit {

    private productService = inject(ProductService);

    products: Product[] = [];

    total = 0;

    loading = false;

    error = "";

    ngOnInit(): void {
        this.loadProducts();
    }

    loadProducts(): void {

        this.loading = true;

        this.error = "";

        this.productService.getProducts()
            .subscribe({

                next: (response: ProductsResponse) => {

                    this.products = response.products;

                    this.total = response.total;

                    this.loading = false;

                },

                error: (error) => {

                    console.error(error);

                    this.error =
                        'No se han podido cargar los productos.';

                    this.loading = false;
                }
            });
    }
}

```

```
    }  
    });  
  }  
}
```

Observación importante sobre Standalone

Aquí tenemos:

```
imports: [  
  CommonModule,  
  
  IonHeader,  
  IonToolbar,  
  IonTitle,  
  ...  
]
```

Esto significa que **el propio componente sabe qué elementos utiliza**.

Por ejemplo:

```
<ion-spinner>
```

requiere:

IonSpinner

Y:

*ngIf

requiere las funcionalidades correspondientes de Angular.

Si utilizamos **CommonModule**, tendremos disponibles las directivas tradicionales como:

```
*ngIf  
*ngFor
```

Una alternativa todavía más moderna

Podemos aprovechar la práctica para introducir la sintaxis moderna de control de flujo de Angular:

```
@if (loading) {  
  ...  
}
```

```
@if (error) {  
  ...  
}
```

```
@for (product of products; track product.id) {  
  ...  
}
```

En ese caso ya no necesitamos `CommonModule` para `*ngIf/*ngFor`.

Para DAM2 recomiendo esta opción, porque permite enseñar Angular actual y evita mezclar sintaxis antigua y moderna.

Por tanto, podemos hacer que la actividad utilice:

```
@if  
@for
```

en lugar de:

```
*ngIf  
*ngFor
```

Tabla utilizando Angular moderno

Nuestro `productos.page.html` quedaría:

```
<ion-header>

  <ion-toolbar>

    <ion-buttons slot="start">

      <ion-back-button
        defaultHref="/inicio">
      </ion-back-button>

    </ion-buttons>

    <ion-title>
      Productos
    </ion-title>

  </ion-toolbar>

</ion-header>

<ion-content class="ion-padding">

  <h1>Listado de productos</h1>

  @if (loading) {

    <div class="loading">

      <ion-spinner></ion-spinner>

      <p>
        Cargando productos...
      </p>

    </div>

  }

  @if (error) {

    <ion-card>
```

```

<ion-card-header>

  <ion-card-title>
    Error
  </ion-card-title>

</ion-card-header>

<ion-card-content>

  <p>
    {{ error }}
  </p>

  <ion-button
    (click)="loadProducts()">

    Reintentar

  </ion-button>

</ion-card-content>

</ion-card>

}

@if (!loading && !error) {

  <p>
    Productos cargados:
    <strong>{{ products.length }}</strong>
  </p>

  <div class="table-container">

    <table>

      <thead>

        <tr>
          <th>ID</th>
          <th>Producto</th>
          <th>Categoría</th>
          <th>Marca</th>
          <th>Precio</th>

```

```
    <th>Valoración</th>
    <th>Stock</th>
  </tr>
```

```
</thead>
```

```
<tbody>
```

```
@for (
  product of products;
  track product.id
) {
```

```
  <tr>
```

```
    <td>
      {{ product.id }}
    </td>
```

```
    <td>
```

```
      <div class="product">
```

```
        <img
          [src]="product.thumbnail"
          [alt]="product.title">
```

```
        <span>
          {{ product.title }}
        </span>
```

```
      </div>
```

```
    </td>
```

```
    <td>
      {{ product.category }}
    </td>
```

```
    <td>
      {{ product.brand || 'Sin marca' }}
    </td>
```

```
    <td>
      {{ product.price | currency:'EUR' }}
    </td>
```

```
    <td>
```

```
        ★ {{ product.rating }}
    </td>

    <td>
        {{ product.stock }}
    </td>

</tr>

}

</tbody>

</table>

</div>

}

</ion-content>
```

¿Qué cambia con **@for**?

Anteriormente teníamos:

```
<tr *ngFor="let product of products">
```

Ahora:

```
@for (  
  product of products;  
  track product.id  
) {
```

Imports de la página Productos

Al utilizar **@if**, **@for** y el pipe **currency**, tenemos que prestar atención a los imports.

Una versión clara sería:

```
import {  
  Component,  
  OnInit,  
  inject  
} from '@angular/core';
```

```
import {  
  CurrencyPipe  
} from '@angular/common';
```

```
import {  
  IonHeader,  
  IonToolbar,  
  IonTitle,  
  IonContent,  
  IonButtons,  
  IonBackButton,  
  IonSpinner,  
  IonCard,  
  IonCardHeader,  
  IonCardTitle,  
  IonCardContent,  
  IonButton  
} from '@ionic/angular/standalone';
```

```
import {  
  Product,
```

```

    ProductsResponse
  } from '../models/product.model';

import {
  ProductService
} from '../services/product.service';

Y:

@Component({
  selector: 'app-productos',

  templateUrl: './productos.page.html',

  styleUrls: ['./productos.page.scss'],

  standalone: true,

  imports: [

    CurrencyPipe,

    IonHeader,
    IonToolbar,
    IonTitle,
    IonContent,
    IonButtons,
    IonBackButton,
    IonSpinner,
    IonCard,
    IonCardHeader,
    IonCardTitle,
    IonCardContent,
    IonButton

  ]
})

```

Estilos

productos.page.scss:

```
.loading {  
  text-align: center;  
  padding: 40px;  
}
```

```
.table-container {  
  width: 100%;  
  overflow-x: auto;  
}
```

```
table {  
  width: 100%;  
  min-width: 900px;  
  border-collapse: collapse;  
}
```

```
th,  
td {  
  padding: 12px;  
  border-bottom: 1px solid #ddd;  
  text-align: left;  
}
```

```
th {  
  background: var(--ion-color-primary);  
  color: white;  
}
```

```
.product {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
}
```

```
.product img {  
  width: 50px;  
  height: 50px;  
  object-fit: contain;  
}
```

Navegación

Para el botón:

```
<ion-button routerLink="/productos">
```

necesitamos importar:

```
import { RouterLink } from '@angular/router';
```

y añadir:

```
imports: [  
  RouterLink,  
  
  IonHeader,  
  IonToolbar,  
  IonTitle,  
  IonContent,  
  IonButton  
]
```

Volver a Inicio

En productos:

```
<ion-back-button  
  defaultHref="/inicio">  
</ion-back-button>
```

y en el componente:

```
IonButtons,  
IonBackButton
```

dentro de `imports`.

Vercel

La configuración Standalone **no cambia el principio del despliegue**, pero debemos comprobar especialmente las rutas SPA.

El proyecto genera una aplicación Angular que utiliza el router del lado cliente.

Por tanto, después de desplegar debemos comprobar:

<https://TU-APP.vercel.app/>

y también directamente:

<https://TU-APP.vercel.app/productos>

Si la navegación funciona desde Inicio pero falla al recargar [/productos](#), habrá que revisar la configuración de rewrites de Vercel.

Git: rama desarrollo

Cuando la aplicación funcione:

```
git init
git add .
git commit -m "Creación inicial de aplicación Ionic Standalone"
```

Creamos:

```
git checkout -b desarrollo
```

Comprobamos:

```
git branch --show-current
```

Debe aparecer:

desarrollo

GitHub

Creamos el repositorio:

dam2-productos

Añadimos el remoto:

```
git remote add origin https://github.com/USUARIO/dam2-productos.git
```

Y publicamos:

```
git push -u origin desarrollo
```

El profesor debe poder seleccionar en GitHub:

Branch → desarrollo

y encontrar allí la versión entregada.

Despliegue en Vercel

En Vercel:

Add New Project



Import Git Repository



dam2-productos

Seleccionamos como rama de producción:

desarrollo

y realizamos el despliegue.

Checklist

Angular Standalone

- No existe `app.module.ts`.
- Los componentes son Standalone.
- Los imports están declarados en los componentes.
- Se utiliza `app.config.ts`.
- Se utiliza `provideHttpClient()`.

API

- Se consume `https://dummyjson.com/products`.
- Existe `Product`.
- Existe `ProductsResponse`.
- Existe `ProductService`.
- Se muestran los datos de la API.

Vista

- Existe una tabla.
- Se muestra ID.
- Se muestra nombre.
- Se muestra categoría.
- Se muestra precio.
- Se muestra stock.
- Se muestra valoración.
- Se muestra imagen.

Navegación

- `/inicio`
- `/productos`
- Redirección `/` → `/inicio`
- Ruta desconocida → `/inicio`
- Navegación Inicio → Productos.
- Navegación Productos → Inicio.

Estados

- Estado de carga.
- Estado correcto.
- Estado de error.
- Botón de reintento.

Git/GitHub

- Repositorio creado.
- Rama `desarrollo`.
- Código subido a `desarrollo`.
- Commits descriptivos.

Vercel

- Proyecto desplegado.
- Rama `desarrollo` utilizada.
- URL pública funcionando.
- `/productos` funciona directamente.
- La API se consume correctamente desde producción.